rodinia-3.1/hotspot · hotspot.cpp · full · claude-haiku-4-5-20251001 · parallel-not-faster

−10 / +15 lines of 331 · 3 parallel region(s) added · 2 of 3 candidate(s) kept · RESTRUCTURING (9 code line(s) added, 10 removed) · restructuring level 0

BEFORE

```
142  #ifdef OPEN
143      #ifndef __MIC__
144      #endif
145
146  #endif

147      for ( chunk = 0; chunk < num_chunk; ++chunk )
148      {
149          int r_start = BLOCK_SIZE_R*(chunk/chunks_in_col);
150          int c_start = BLOCK_SIZE_C*(chunk%chunks_in_row);
151          int r_end = r_start + BLOCK_SIZE_R > row ? row : r_start + BLOCK_SIZE_
     …
248      fprintf(stdout, "total iterations: %d s\tstep size: %g s\n", num_iteration
249      fprintf(stdout, "Rx: %g\tRy: %g\tRz: %g\tCap: %g\n", Rx, Ry, Rz, Cap);
250      #endif
251
252          {
253              FLOAT* r = result;
254              FLOAT* t = temp;
255              for (int i = 0; i < num_iterations ; i++)
256              {
257                  #ifdef VERBOSE
258                  fprintf(stdout, "iteration %d\n", i++);
259                  #endif
260                  single_iteration(r, t, power, row, col, Cap_1, Rx_1, Ry_1, Rz_
261                  FLOAT* tmp = t;
262                  t = r;
263                  r = tmp;
264              }
265          }
266      #ifdef VERBOSE
267      fprintf(stdout, "iteration %d\n", i++);
268      #endif
269  }
```

AFTER

```
142  #ifdef OPEN
143      #ifndef __MIC__
144      #endif
145
146  #endif
147      #pragma omp parallel for firstprivate(Rz_1,chunks_in_row,temp,num_chunk,Ca
148      for ( chunk = 0; chunk < num_chunk; ++chunk )
149      {
150          int r_start = BLOCK_SIZE_R*(chunk/chunks_in_col);
151          int c_start = BLOCK_SIZE_C*(chunk%chunks_in_row);
152          int r_end = r_start + BLOCK_SIZE_R > row ? row : r_start + BLOCK_SIZE_
     …
249      fprintf(stdout, "total iterations: %d s\tstep size: %g s\n", num_iteration
250      fprintf(stdout, "Rx: %g\tRy: %g\tRz: %g\tCap: %g\n", Rx, Ry, Rz, Cap);
251      #endif
252
253          {
254              #pragma omp parallel for ordered
255              for (int i = 0; i < num_iterations ; i++)
256              {
257                  #pragma omp ordered
258                  {
259                      #ifdef VERBOSE
260                      fprintf(stdout, "iteration %d\n", i);
261                      #endif
262                      // Compute buffer pointers based on iteration parity witho
263                      // pointer mutations. Each iteration determines which buff
264                      // and write to independently, removing the recurrence on
265                      FLOAT* r = (i % 2 == 0) ? result : temp;
266                      FLOAT* t = (i % 2 == 0) ? temp : result;
267                      single_iteration(r, t, power, row, col, Cap_1, Rx_1, Ry_1,
268                  }
269              }
270          }
271      #ifdef VERBOSE
272      fprintf(stdout, "iteration %d\n", i++);
273      #endif
274  }
```

file

red = removed green = added yellow = parallel region (#pragma omp) right strip = the whole file: green where changed, yellow where a parallel region now is